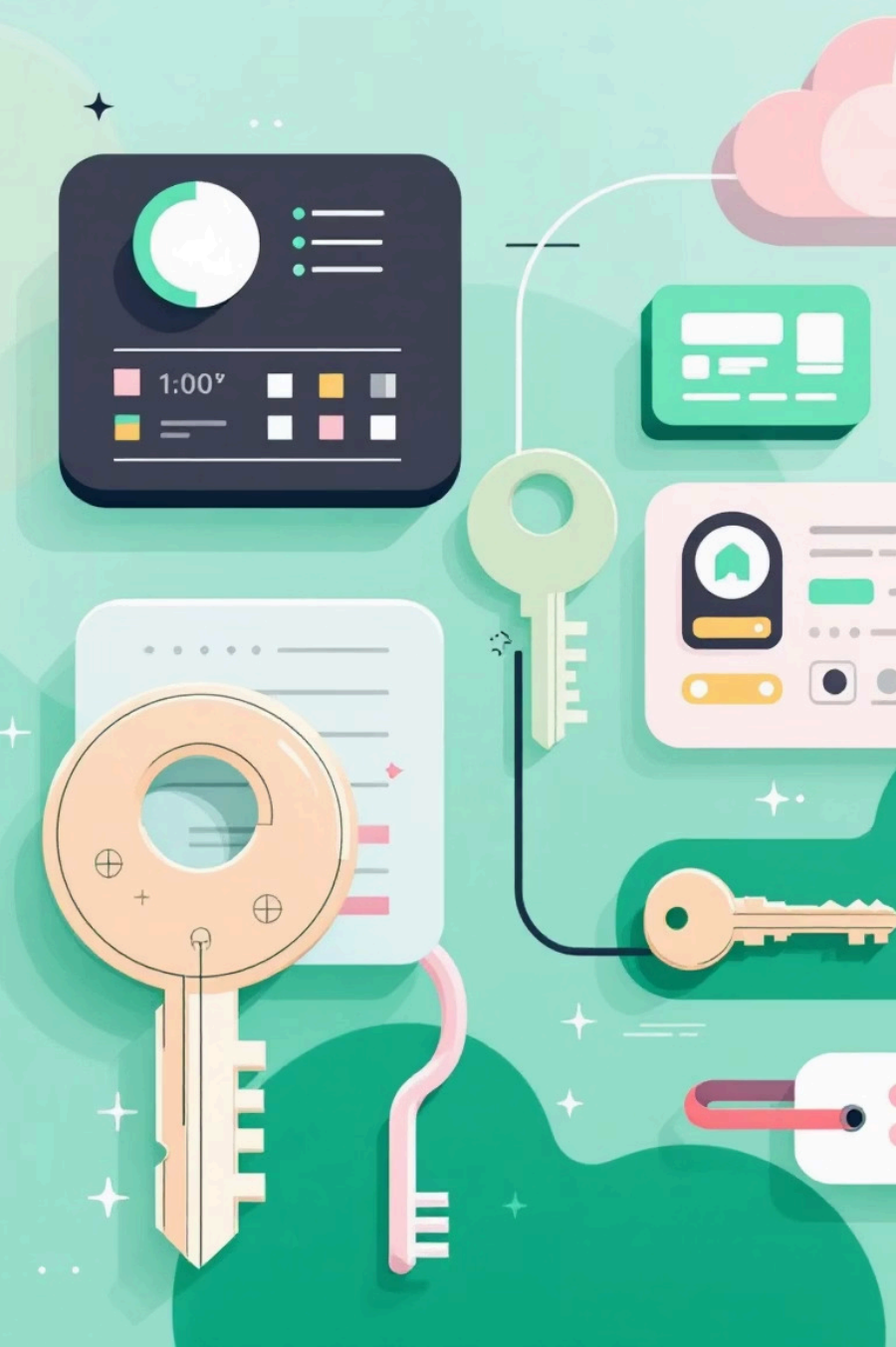




Advanced Topics in Flutter

This presentation delves into Flutter's internal workings, moving beyond basic UI development to explore the core mechanisms that drive its performance and efficiency.



CHAPTER 1

The Three Pillars: Flutter Trees

Flutter's architecture is built upon three interconnected trees, each playing a crucial role in rendering and managing the user interface. Understanding these trees is fundamental to grasping how Flutter efficiently updates and displays UI elements.



1. The Widget Tree

Defines UI Structure

Composed of **Widget objects** that describe the desired appearance of the UI.

Immutable Nature

Widgets are **immutable**, meaning they cannot be changed once created. Any UI change requires a new Widget.

Example

`Text("Hello")` is a simple widget defining text content.

2. The Element Tree

3. The Render Tree

1

Element Tree

Connects widgets to render objects, managing widget state and efficient updates. An Element is a widget instance in the tree.

2

Render Tree

Responsible for layout, painting, positioning, and sizing. This tree directly **draws pixels on the screen**.

How the Trees Work Together



You Write Widgets

Developers define the UI using declarative widgets.



Flutter Creates Elements

Elements are generated to manage the widget instances.



Elements Create RenderObjects

RenderObjects handle the actual rendering details.



Screen is Painted

The final UI is drawn on the device screen.

This layered approach ensures better performance, efficient UI updates, and a clear separation of responsibilities within the Flutter framework.

BuildContext: Widget Location Reference

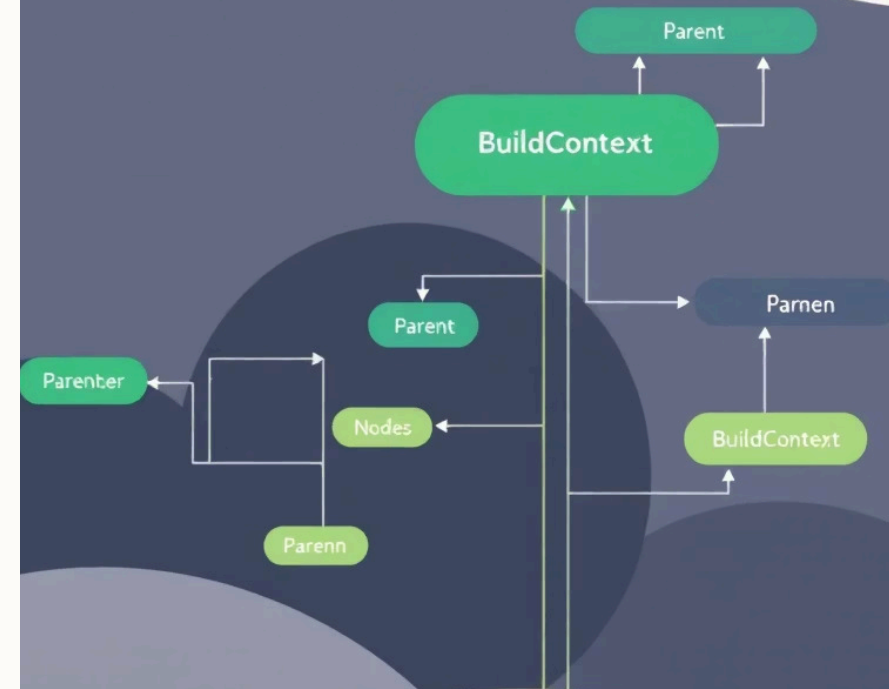
BuildContext is a crucial concept in Flutter, acting as a reference to a widget's specific location within the widget tree. It enables Flutter to efficiently locate parent widgets and access inherited data.

Key Functions

- Find parents
- Access inherited widgets
- Get theme, media query, etc.

Examples

- `Theme.of(context)`
- `MediaQuery.of(context)`
- `context.read()`



BuildContext: Important Rules & Common Mistakes

Important Rule

- ❑ Context works only inside its widget tree.
- Always use context inside `build()` or within callbacks, never before.

Correct Usage

`inside build() or callbacks`

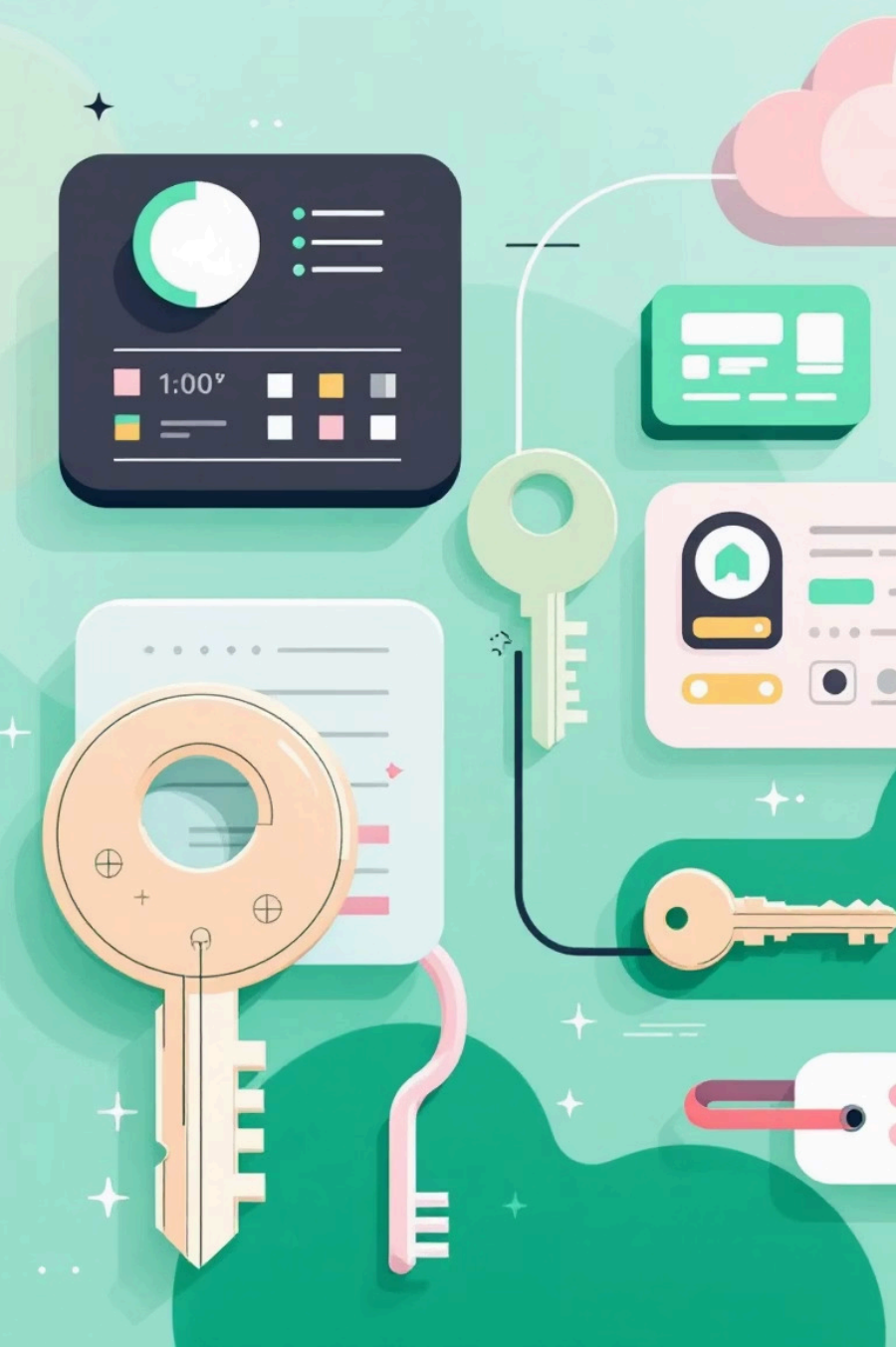
Common Mistake

- ❑ Using context from a widget that is not wrapped with `Provider` / `BlocProvider`, leading to errors.

Incorrect Usage

`use context before build()`





CHAPTER 3

Keys in Flutter: Ensuring Widget Identity

Keys are essential identifiers in Flutter, telling the framework that a specific widget is unique. They are critical for preserving state, preventing UI bugs, and optimizing performance, especially in dynamic lists.

1

What is a Key?

A Key signals to Flutter: "This widget is unique." It helps Flutter correctly identify widgets during UI rebuilds.

2

Why Important?

- Preserve state
- Fix wrong UI updates
- Improve list performance

Keys: Examples and Types

Example Without Key ❌

```
ListView(  
  children: [  
    TextField(),  
    TextField(),  
  ],  
);
```

Without keys, Flutter may mix states, leading to incorrect UI behavior.

Example With Key ✅

```
TextField(key: ValueKey("email"))  
TextField(key: ValueKey("password"))
```

Keys ensure Flutter correctly identifies and manages each widget's state.

Types of Keys



ValueKey



UniqueKey



ObjectKey



GlobalKey (advanced)

const in Flutter: Performance Optimization

The `const` keyword in Flutter signifies that a widget will never change. This allows Flutter to create it once and reuse it, leading to significant performance benefits.

Why Use `const`?

- Better performance
- Fewer rebuilds
- Less memory usage

Important Rule

 Use `const` **whenever possible**. If all parameters are constant, the widget can be constant.



Difference Between `const` and `final`

<code>const</code>	<code>final</code>
Compile time	Runtime
Never changes	Assigned once
Used with widgets	Used with variables